

Lab 7: Abstract class & Interfaces & Exception Handling

1. Abstract Classes and Methods

- **Abstract Method:** A method declaration without implementation (no body)
- public abstract double calculateArea();

```
public abstract double calculateArea();
```

- **Abstract Class:**
 - Contains at least one abstract method
 - Cannot be instantiated (must be inherited)
 - Can contain both abstract and concrete methods

Example:

```
public abstract class Shape {  
    public abstract double calculateArea();  
}
```

- **Rules:**
 - If a class has one abstract method, it must be abstract
 - Subclasses must implement all abstract methods or be abstract themselves

2. Interfaces

- Contains only constants and abstract methods (no constructors)
- Uses implements keyword
- A class can implement multiple interfaces

Example:

```
public interface Payable {  
    double getPaymentAmount();  
}  
  
public class Invoice implements Payable {  
    public double getPaymentAmount() {  
        // implementation  
    }  
}
```

- **Interface Characteristics:**
 - Methods are implicitly public abstract
 - Variables are implicitly static final
 - Represents "provides a" relationship

3. Method Overloading

- Multiple methods with same name but different parameters
- Distinguished by parameter types/number (not return type)

Example:

```
class A {  
    void m1(int x) { ... }  
    void m1(int x, int y) { ... }  
    void m1(double x) { ... }  
}
```

4. ArrayList

- Dynamic array that grows/shrinks as needed

Syntax:

```
ArrayList<String> list = new ArrayList<>();
```

- **Key Operations:**
 - add(), remove(), get(), set()
 - Collections.sort() for sorting

Example:

```
ArrayList<String> fruits = new ArrayList<>();  
fruits.add("Apple");  
fruits.add("Banana");
```

5. Exception Handling

- **Try-Catch Blocks:**

```
try {  
    // code that might throw exception  
} catch (ExceptionType e) {  
    // handle exception  
}
```

- **Common Exceptions:**
 - ArithmeticException (divide by zero)
 - NullPointerException (accessing null reference)
 - IndexOutOfBoundsException (invalid array index)
- **Exception Propagation:** Exceptions can propagate up the call stack

Key Differences: Abstract Class vs Interface

Feature	Abstract Class	Interface
Methods	Can have both abstract and concrete	Only abstract (until Java 8)
Variables	Any type	Only constants (static final)
Inheritance	Single inheritance	Multiple implementations
Constructors	Can have constructors	No constructors
Relationship	"Is-a"	"Provides-a"

Problem 1: Bank Account System with Abstract Classes (5 Points)

Description:

Design a banking system using abstract classes and interfaces to model different account types with transaction capabilities.

Requirements:

- UML Class Diagram (1 Point)**
 - Abstract class: BankAccount
 - Attributes: accountNumber (String), balance (double)
 - Methods: deposit(amount), withdraw(amount) (abstract), displayBalance()
 - Interface: InterestBearing
 - Method: calculateInterest()
 - Subclasses:
 - SavingsAccount (implements InterestBearing)
 - CheckingAccount
- Abstract Class & Interface Implementation (1 Point)**
 - BankAccount:
 - Constructor to set accountNumber and balance.
 - Concrete deposit() method.
 - Abstract withdraw().
 - InterestBearing:
 - Declare calculateInterest().
- Subclass Definitions (1 Point)**
 - SavingsAccount:
 - Override withdraw() (no overdraft).
 - Implement calculateInterest() (5% of balance).
 - CheckingAccount:
 - Override withdraw() (allow overdraft up to -1000).
- ArrayList & Exception Handling (1 Point)**
 - Create an ArrayList<BankAccount> to store 2 accounts (1 Savings, 1 Checking).
 - Use a try-catch block to handle ArithmeticException in withdraw() if balance is insufficient.
- Testing (1 Point)**
 - Deposit 500, withdraw 200 from SavingsAccount.
 - Attempt to withdraw 1500 from CheckingAccount (catch exception).
 - Print balances and interest (for SavingsAccount).

Expected Output:

Savings Balance: 300.0 | Interest: 15.0
Checking Balance: -1000.0
Error: Overdraft limit exceeded!

Problem 2: University Course Registration with Interfaces (5 Points)

Description:

Model a course registration system using interfaces and polymorphism, with exception handling for enrollment limits.

Requirements:

1. **UML Class Diagram** (1 Point)
 - Interface: Enrollable
 - Methods: enroll(Student), drop(Student)
 - Class: Student
 - Attributes: studentId (int), name (String)
 - Class: Course (implements Enrollable)
 - Attributes: courseCode (String), maxStudents (int), students (ArrayList<Student>)
2. **Interface & Class Implementation** (1 Point)
 - Enrollable:
 - Declare enroll() and drop().
 - Course:
 - Constructor with courseCode and maxStudents.
 - Implement enroll() (add to students ArrayList).
3. **Polymorphism & Exception Handling** (1 Point)
 - In Course.enroll(), throw a custom EnrollmentException if students.size() >= maxStudents.
 - Catch the exception in the test code.
4. **Method Overloading** (1 Point)
 - Add overloaded enroll() in Course:
 - enroll(Student) (single student).
 - enroll(List<Student>) (batch enrollment).
5. **Testing** (1 Point)
 - Create:
 - A Course ("CS101", maxStudents=2).
 - 3 Student objects.
 - Enroll 2 students (success), then a 3rd (catch exception).
 - Use the overloaded enroll() to add a list of students.

Expected Output:

Enrolled Ali in CS101.
Enrolled Sara in CS101.
Error: Course CS101 is full!
Batch enrollment succeeded for 1 students.